САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Домашнее задание №4 «Технический дизайн микросервисной архитектуры»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкенд-приложение фото-соцсети (реализованное в ЛР1) на микросервисы, придерживаясь концепции **database-per-service**. Спроектировать микросервисную архитектуру, отразить взаимодействие сервисов, разделить БД на обособленные хранилища, описать внутренние эндпоинты и события, а также сформировать пошаговый план миграции монолита в микросервисную архитектуру.

Ход работы

1. Анализ текущего монолита

В данный момент есть 7 сущностей и 5 контроллеров:

Сущность	**Контроллер**
User	AuthController, UserController
Post	PostController
Like	PostController
Favorite	PostController
Comment	CommentController
Follow	UserController
Notification	NotificationController

Все они находятся в одном приложении и одной БД (SQLite).
Задача в том, чтобы разбить на независимые сервисы с отдельной БД у каждого.

2. Предлагаемое разбиение и описание микросервисов

Я решил сгруппировать 4 микросервиса:

1. **Auth Service –** сервис аутентификации и пользователей
   Отвечает за:
   - Регистрацию
   - Логин (выдача JWT)
   - Профиль пользователя
   - Загрузку аватара

   БД: auth_db
   users (id, username, email, passwordHash, avatarPath, createdAt)

Если аутентификация упадёт, остальные сервисы благодаря переходу на микросервисную архитектуру смогут принимать запросы по уже выданным токенам и кэшированным данным.

2. **Post Service –** сервис постов

Отвечает за:

- Создание постов с фотографиями
- Ленту постов
- Получение поста по ID
- Удаление постов
- Лайки
- Избранное

БД: post_db

posts (id, userId, originalPhotoPath, previewPhotoPath, description, createdAt)

likes (id, userId, postId, createdAt)

favorites (id, userId, postId, createdAt)

База данных хранит только userId, для отображения автора поста она обращается к Auth Service.

3. **Comment Service** – сервис комментариев

Отвечает за:

- Добавление комментариев
- Получение комментариев поста
- Удаление комментариев

БД: comment_db

comments (id, userId, postId, text, photoPath, createdAt)

Комментарии, как отдельный поток данных, обычно появляются сильно быстрее постов, так что их вынесение в отдельный сервис позволит масштабировать сервис независимо.

4. **Social Service** – сервис для «соцсетевой» части приложения

Отвечает за:

- Подписки и отписки
- Уведомления
- Хранение графа подписок

БД: social_db

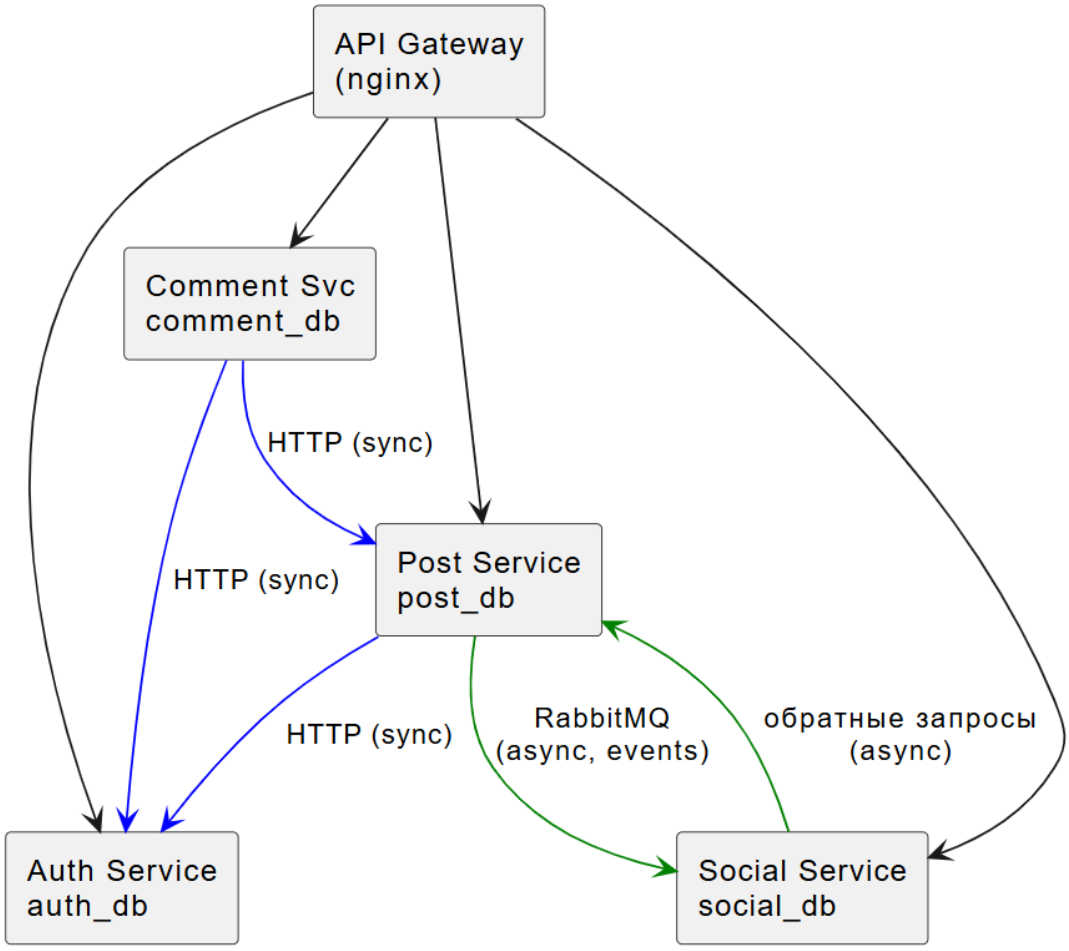
follows (id, followerId, followingId, createdAt)

notifications (id, userId, type, relatedUserId, relatedPostId, isRead, createdAt)

Уведомления и подписки тесно связаны – грубо говоря, подписки всегда порождают уведомления, также это позволяет избежать избыточности в делении монолита на микросервисы.

3. Микросервисная архитектура и взаимодействие

Схема архитектуры в общих чертах:



Синхронное взаимодействие (HTTP/REST) используется там, где нужен немедленный ответ:

- **Post Service → Auth Service** - получить данные автора поста (username, avatarPath)
- **Comment Service → Auth Service** - получить данные автора комментария
- **Comment Service → Post Service** - проверить, что пост существует, прежде чем добавить комментарий
- **Post Service → Social Service** - получить список подписок пользователя для формирования ленты

Асинхронное взаимодействие (RabbitMQ) используется для событий, соответственно, не требующих немедленного ответа:

- **Post Service → Social Service** – поставить лайк
- **Comment Service → Social Service** – создать комментарий
- **Auth Service → Social Service** – зарегистрировать нового пользователя
-

4. Разделение базы данных (database-per-service)

Сервис	БД	Таблицы
Auth Service	auth_db	users
Post Service	post_db	posts, likes, favorites
Comment Service	comment_db	comments
Social Service	social_db	follows, notifications

Принципы деления:

1. **Никаких внешних ключей между БД -** поля userId, postId хранятся как обычные числовые значения.
2. **Согласованность через события -** например, при удалении поста Post Service публикует событие post.deleted, на которое подписан Comment Service (удаляет комментарии) и Social Service (удаляет уведомления).
3. **Eventual consistency -** данные между сервисами синхронизируются с небольшой задержкой, что допустимо для социальной сети.
4. **Дублирование допустимо -** например, Post Service может локально кэшировать username автора для быстрой отдачи ленты.

5. Внутренние эндпоинты (синхронное взаимодействие)

Ниже описаны внутренние REST-эндпоинты, которые используются только для межсервисного взаимодействия. Полная OpenAPI-спецификация будет реализована в рамках ЛР2.

Auth Service

Метод	Эндпоинт	Описание	Ответ
GET	/internal/users/:id	Получить данные пользователя по ID	{id, username, email, avatarPath}
POST	/internal/users/batch	Получить данные нескольких пользователей	[{id, username, avatarPath}, ...]
GET	/internal/users/:id/exists	Проверить существование пользователя	{exists: true/false}
POST	/internal/auth/verify	Проверить JWT-токен	{userId, valid: true}

Post Service

Метод	Эндпоинт	Описание	Ответ
GET	/internal/posts/:id	Получить пост по ID	{id, userId, description, ...}
GET	/internal/posts/:id/exists	Проверить существование поста	{exists: true/false}
GET	/internal/posts/by-user/:userId	Список постов пользователя	[{id, ...}]
GET	/internal/posts/by-ids?ids=1,2,3	Список постов по списку ID	[{id, ...}]
DELETE	/internal/posts/by-user/:userId	Удалить все посты пользователя	{deleted: N}

Comment Service

Метод	Эндпоинт	Описание	Ответ
GET	/internal/comments/by-post/:postId	Комментарии к посту	[{id, ...}]
GET	/internal/comments/count/:postId	Количество комментариев	{count: N}
DELETE	/internal/comments/by-post/:postId	Удалить все комментарии поста	{deleted: N}
DELETE	/internal/comments/by-user/:userId	Удалить все комментарии пользователя	{deleted: N}

Social Service

Метод	Эндпоинт	Описание	Ответ
GET	/internal/follows/followers/:userId	Подписчики пользователя	[{followerId, ...}]
GET	/internal/follows/following/:userId	Подписки пользователя	[{followingId, ...}]
POST	/internal/follows/check	Проверить подписку	{exists: true/false}
DELETE	/internal/notifications/by-post/:postId	Удалить уведомления по посту	{deleted: N}

Примеры запросов и ответов
Пример 1. Post Service запрашивает данные автора поста

Запрос:
GET http://auth-service:3001/internal/users/5

Успешный ответ (200 OK):
```
{
  "id": 5,
  "username": "bogdan",
  "avatarPath": "/uploads/avatars/5_avatar.jpg"
}
```

Ошибка (404 Not Found):
```
{
  "message": "User not found"
}
```

Пример 2. Comment Service проверяет существование поста
Запрос:
GET http://post-service:3002/internal/posts/10/exists

Успешный ответ (200 OK):
```
{
  "exists": true
}
```

Пример 3. Ошибка авторизации между сервисами
Запрос:
GET http://auth-service:3001/internal/users/5
Authorization: Bearer <invalid-service-token>

Ответ (401 Unauthorized):
```
{
  "message": "Invalid service token"
}
```

Все внутренние эндпоинты защищены сервисным токеном, чтобы исключить доступ извне.

6. Асинхронное взаимодействие (RabbitMQ)
Для событий, не требующих немедленного ответа, используется брокер сообщений RabbitMQ.

Обменники (exchanges) и очереди (queues)

Обменник	**Тип**	**Очереди-подписчики**	**Событие**
post.events	topic	social.like.created, social.post.deleted	События постов
comment.events	topic	social.comment.created	События комментариев
user.events	topic	social.user.created	События пользователей

Описание событий

Событие	**Источник**	**Потребители**	**Payload**
like.created	Post Service	Social Service	{userId, postId, postOwnerId, createdAt}
like.removed	Post Service	Social Service	{userId, postId}
comment.created	Comment Service	Social Service	{commentId, userId, postId, postOwnerId, hasPhoto, createdAt}
post.deleted	Post Service	Comment Service, Social Service	{postId, userId}
user.created	Auth Service	Social Service	{userId, username}
user.deleted	Auth Service	Post Service, Comment Service, Social Service	{userId}

Пример обработки события:
Пользователь А лайкает пост пользователя В:

1. **Post Service** обрабатывает запрос POST /posts/10/like, сохраняет лайк в post_db и публикует событие like.created в обменник post.events.
2. **Social Service** получает это событие из очереди, проверяет, что userId != postOwnerId, и создаёт запись в notifications (тип like).
3. Пользователь В при следующем запросе GET /notifications получает уведомление.

Такой подход позволяет Post Service не зависеть от Social Service, и если последний временно недоступен, события буферизуются в RabbitMQ и обрабатываются после восстановления.

7. API Gateway (nginx)

Для маршрутизации внешних запросов к нужным микросервисам используется nginx в режиме обратного прокси. Он выполняет следующие функции:

- **Маршрутизация** по префиксу URL:
  - /api/auth/* → Auth Service
  - /api/posts/* → Post Service
  - /api/comments/* → Comment Service
  - /api/users/follow/*, /api/notifications/* → Social Service
- **Балансировка нагрузки** между экземплярами сервисов (round-robin).
- **Отдача статики** (/uploads/*) - либо через отдельный сервис, либо через общий том.
- **Терминация TLS** - единая точка входа HTTPS.
- **Блокировка доступа к** /internal/* - внутренние эндпоинты недоступны снаружи.
- **Ограничение размера запроса** - для загрузки файлов.
- **Логирование** всех входящих запросов.

Пример конфигурации (фрагмент):

```
upstream auth_service    { server auth-service:3001; }
upstream post_service    { server post-service:3002; }
upstream comment_service { server comment-service:3003; }
upstream social_service  { server social-service:3004; }

server {
  listen 80;

  location /api/auth/       { proxy_pass http://auth_service/; }
  location /api/posts/      { proxy_pass http://post_service/; }
  location /api/comments/   { proxy_pass http://comment_service/; }
  location /api/notifications/ { proxy_pass http://social_service/notifications/; }
  location /api/users/follow/  { proxy_pass http://social_service/follows/; }

  location /uploads/ { alias /var/www/uploads/; }

  location /internal/ { return 403; }
}
```

8. Пошаговый план миграции монолита в микросервисы

Этап 1. Подготовка

1. Разделить репозиторий на отдельные директории для каждого сервиса.
2. Создать общий пакет для типов DTO и утилит.
3. Определить контракты (OpenAPI + события RabbitMQ) между сервисами.

Этап 2. Выделение Auth Service

1. Вынести модели User и логику аутентификации.
2. Создать auth_db.
3. Реализовать /internal/users/:id.
4. Остальные сервисы пока обращаются к монолиту; интерфейс не меняется.

Этап 3. Выделение Post Service

1. Вынести Post, Like, Favorite.
2. Создать post_db.
3. Реализовать /internal/posts/:id, /internal/posts/:id/exists.
4. Настроить вызовы к Auth Service для получения авторов.

Этап 4. Выделение Comment Service

1. Вынести Comment.
2. Создать comment_db.
3. Реализовать /internal/comments/by-post/:postId.
4. Настроить вызовы к Auth и Post Service.
5.

Этап 5. Выделение Social Service

1. Вынести Follow, Notification.
2. Создать social_db.
3. Настроить RabbitMQ, публиковать события like.created, comment.created.

Этап 6. Внедрение API Gateway

1. Установить nginx, настроить маршрутизацию.
2. Закрыть /internal/* от внешнего доступа.
3. Настроить TLS.

Этап 7. Переход на Docker Compose

1. Написать Dockerfile для каждого сервиса.
2. Создать общий docker-compose.yml с RabbitMQ, nginx и 4 сервисами.
3. Настроить сеть между контейнерами.

Этап 8. Тестирование и отладка

1. Прогнать существующую Postman-коллекцию (ДЗ3) против микросервисной архитектуры.
2. Убедиться, что все сценарии работают.
3. Проверить корректность обработки событий RabbitMQ.

Этап 9. Отказ от монолита

1. Убедиться, что все эндпоинты обрабатываются микросервисами.
2. Удалить монолитный код.

Вывод

В ходе выполнения ДЗ4 спроектирована микросервисная архитектура фото-соцсети. Монолитное приложение разделено на четыре сервиса: **Auth Service**, **Post Service**, **Comment Service**, **Social Service**. Каждый сервис имеет собственную БД, что соответствует принципу database-per-service. Определены два способа взаимодействия: синхронный (HTTP/REST через внутренние эндпоинты /internal/*) и асинхронный (RabbitMQ для событий). Спроектированы внутренние эндпоинты для межсервисного взаимодействия и описаны возможные ошибки. Внедрение API Gateway на nginx обеспечивает единую точку входа и защиту внутренних эндпоинтов. Составлен пошаговый план миграции монолита в микросервисную архитектуру, который будет реализован в рамках ЛР2. Спроектированная архитектура удовлетворяет требованиям масштабируемости, отказоустойчивости и независимого развёртывания сервисов, сохраняя при этом функциональность исходного монолитного приложения.